claude-task-templates

A repo-local memory layer for long-running Claude Code sessions and multi-session orchestration.

Version 1 · 2026-04-20 · github.com/ericwang19832003/claude-task-templates

Contents

1. What this is and why it exists
2. The five goals
3. How it fits with existing Claude Code memory
4. What was implemented
5. Install
6. Daily use — five commands you actually run
7. The six task files in detail
8. The lifecycle — init, checkpoint, takeover, fold, archive
9. The orchestrator API
10. Anti-patterns to avoid
11. Example: the live Foundation task
12. Where things live (file map)

1. What this is and why it exists

Claude Code conversations end. Sometimes deliberately (the user runs `/clear` or starts a fresh session), sometimes because context grows too large to be useful. When a session ends mid-work, the next agent — possibly a different model, possibly on a different machine — usually has to reconstruct what was happening, re-litigate decisions, and re-learn the constraints. That's expensive and error-prone.

This system is a small set of Markdown templates plus a shell script that creates a per-task working memory folder inside any repo. An arriving agent reads three files (Status, Brief, Next) and is up to speed in roughly five minutes. Old context can be safely thrown away because the important state lives on disk.

2. The five goals

1. A session can be reset (or `/clear`-ed) at any time without losing important state.
2. Context doesn't grow unboundedly and degrade model performance.
3. A brand-new agent can take over a task within **five minutes** by reading the memory files.
4. An orchestrator (script, scheduled trigger, dashboard) can read task status programmatically.
5. Repeated work and back-and-forth on the same problem across sessions is reduced.

3. How it fits with existing Claude Code memory

Three coexisting layers, each with a different timescale:

LAYER	WHERE	LIFETIME	PURPOSE
Auto-memory EXISTING	`~/.claude/projects/ … /memory/`	Months to years	User profile, feedback rules, project background, references.
Task memory THIS	`<repo>/.claude-task/<slug>/`	Hours to weeks	Cross-session handoff: decisions, log, artifacts.
Harness tasks EXISTING	`~/.claude/tasks/<session>/<id>.json`	One session	Within-session step tracking via `TaskCreate` / `TaskUpdate`.

This system only addresses the middle layer. The other two stay as-is and are not replaced.

4. What was implemented

One new public GitHub repo and one update to AdhocPrintStudio.

Repo: claude-task-templates (new)

```
claude-task-templates/
  bin/
    claude-task-init            one shell script that scaffolds a task folder
  templates/
    README.md.tmpl              .claude-task/README.md (committed in target repo)
    ACTIVE.tmpl                 one-line file naming the active task slug
    task/
      STATUS.md.tmpl
      BRIEF.md.tmpl
      NEXT.md.tmpl
      DECISIONS.md.tmpl
      LOG.md.tmpl
      ARTIFACTS.md.tmpl
  snippets/
    CLAUDE.md.snippet           block to append to a target repo CLAUDE.md
    gitignore.snippet           two-line gitignore pattern
  docs/
    specs/
      2026-04-20-design.md      full design specification
      2026-04-20-plan.md        implementation plan
    guide.html                  this document, source of the PDF
  README.md
  LICENSE                       MIT
```

Repo: AdhocPrintStudio (updated)

The existing repo now opts in to the system. `.gitignore` gained the two-line pattern and `.claude-task/README.md` was committed (everything else under `.claude-task/` is local-only). The first real task folder, `.claude-task/ui-redesign-foundation/`, was populated with the actual state of the in-flight UI redesign Foundation work as a dogfood test.

5. Install

One-time, in Git Bash:

```
git clone https://github.com/ericwang19832003/claude-task-templates.git ~/claude-task-template
echo 'export PATH="$HOME/claude-task-templates/bin:$PATH"' >> ~/.bashrc
exec $SHELL
```

On this machine the repo is already at `C:\Users\Eric\claude-task-templates\`, which is `~/claude-task-templates` from Git Bash — only the `PATH` export and shell reload are still needed.

Verify the install:

```
which claude-task-init
# expected: /c/Users/Eric/claude-task-templates/bin/claude-task-init
```

6. Daily use — five commands you actually run

GOAL	COMMAND OR ACTION
Start a new task	`claude-task-init my-task --title "What I'm working on"`
See the active task	`cat .claude-task/ACTIVE`
Take over an existing task (in a fresh session)	`cd .claude-task/$(cat .claude-task/ACTIVE)/` Then read `STATUS.md` → `BRIEF.md` → `NEXT.md`, skim `DECISIONS.md`.
Checkpoint before a reset or natural pause	Edit `STATUS.md` (rewrite "Where we are", bump `updated`), overwrite `NEXT.md` with literal next 1–3 steps, append a `LOG.md` entry. ≤2 minutes.
Read every task's status programmatically	Python or `yq` snippet — see §9.

> **The discipline that makes the rest work:** checkpoint at every natural pause — finishing a phase, hitting a blocker, switching focus, before you sense context bloat. A two-minute checkpoint every hour beats a twenty-minute reconstruction after a `/clear`.

7. The six task files in detail

Every task folder contains exactly these six files. Predictable structure means an arriving agent never wonders whether a file exists.

FILE	JOB	MUTATION	READ BY TAKEOVER?
`STATUS.md`	Snapshot of current state plus YAML frontmatter	Overwritten frequently	Yes (always)
`BRIEF.md`	Goal, scope, success criteria, constraints	Write-once at init	Yes (always)
`NEXT.md`	Concrete next 1–3 steps	Overwritten at every checkpoint	Yes (always)
`DECISIONS.md`	Irreversible choices with why-lines	Append-only, newest at top	Skim
`LOG.md`	Append-only journal of significant events	Append; fold when >500 lines	Lazy / on demand
`ARTIFACTS.md`	Pointers to files, commits, PRs, URLs produced	Append-only	On demand

STATUS.md — the orchestrator contract

The YAML frontmatter is the contract that scripts read. Field reference:

FIELD	TYPE	MEANING
`task_id`	string (slug)	Stable identity, never renamed.
`title`	string	One-line human title.
`status`	enum	`not_started` · `in_progress` · `blocked` · `awaiting_review` · `done` · `abandoned`
`owner`	string	Current owner (model name or human name) or `unassigned`.
`session_id`	UUID or null	Which Claude session is on it now.
`created` / `updated`	ISO 8601 UTC	Bump `updated` on every edit.
`parent_task`	slug or null	For sub-tasks of larger work.
`blockers_open`	integer	Quick dashboard signal.
`next_action_owner`	`claude` · `human` · `unknown`	Who needs to do the next thing.

8. The lifecycle — init, checkpoint, takeover, fold, archive

Init — create a new task

Trigger: a piece of work expected to span more than one session, or that produces enough state to need handoff. *A typo fix doesn't need a folder.*

```
claude-task-init <slug> --title "…" [--owner <name>] [--parent <slug>]
```

The script picks a slug, creates `.claude-task/<slug>/` with all six files, fills timestamps, sets `.claude-task/ACTIVE`, and (if needed) drops the `README.md` and updates `.gitignore`. Then by hand: fill `BRIEF.md`, write the first `NEXT.md`, set status to `in_progress`.

Checkpoint — before reset, /clear, or handoff

1. Update `STATUS.md`: rewrite "Where we are" (≤80 words), bump `updated`, update `status` if changed, update `blockers_open`, paste last 3 LOG entries into "Recent log excerpt".
2. Overwrite `NEXT.md` with the literal next 1–3 things. Be specific: file paths, commands, hashes, URLs.
3. Append a `LOG.md` entry summarising what happened since the last checkpoint.
4. If a new irreversible choice was made, append to `DECISIONS.md`.
5. If a new artifact was produced (commit, PR, file, URL), append to `ARTIFACTS.md`.

6. If `LOG.md` exceeds 500 lines, run Fold.

Takeover — fresh agent picking up

Goal: working knowledge in five minutes.

STEP	TIME	ACTION
1	1 sec	`cat .claude-task/ACTIVE`
2	1 sec	`cd .claude-task/$(cat .claude-task/ACTIVE)/`
3	30 sec	Read `STATUS.md` — frontmatter + body.
4	1 min	Read `BRIEF.md` — goal, scope, success.
5	30 sec	Read `NEXT.md` — concrete next step.
6	2 min	Skim `DECISIONS.md` (newest 5–10 entries) so you don't re-litigate.
7	30 sec	Update `STATUS.md` frontmatter: new `owner`, new `session_id`, bump `updated`.
8	—	Optionally open `LOG.md` only if confused.
9	—	Mirror `NEXT.md` steps into harness `TaskCreate` calls and start.

`LOG.md` **is opt-in.** That single design choice is what makes a five-minute takeover achievable.

Fold — compress LOG.md when over 500 lines

1. Identify the 200 oldest entries (bottom of file).
2. Write a `## Summarized through <date>` block: 5–15 bullets, one per major arc, preserving commit hashes / PR numbers / key file paths.
3. Insert that block at the top of `LOG.md` (above the 300 most recent entries).
4. Delete the 200 original entries that were folded.

A folded LOG never grows the summary block beyond about 30 lines — re-folds compress prior summaries with the new oldest 200. Bounded forever.

Archive — when status flips to done or abandoned

1. Final checkpoint — last STATUS update, last LOG entry, success-criteria check-off.
2. `mv .claude-task/<slug> .claude-task/_archive/<slug>`
3. If `.claude-task/ACTIVE` pointed to `<slug>`, clear it or update it to the next task.

Archived tasks are read-only — preserved for history, grep, and audit.

Where the protocols live so agents follow them

Append the snippet at `~/claude-task-templates/snippets/CLAUDE.md.snippet` to your repo's `CLAUDE.md`. Then every Claude Code session in that repo sees the protocol at startup and knows to look at `.claude-task/ACTIVE` first.

9. The orchestrator API

Python — no dependencies beyond PyYAML

```
import yaml, pathlib, re
for f in pathlib.Path('.claude-task').glob('*/STATUS.md'):
    text = f.read_text()
    m = re.match(r'---\n(.*?)\n---', text, re.S)
    if m:
        fm = yaml.safe_load(m.group(1))
        print(fm['task_id'], fm['status'], fm['owner'], fm['updated'])
```

Bash plus yq

```
for d in .claude-task/*/; do
  [ -f "$d/STATUS.md" ] || continue
  yq -f extract '"\(.task_id)\t\(.status)\t\(.owner)\t\(.updated)"' "$d/STATUS.md"
done
```

Use cases this enables

- A `/loop` watcher that pings the user when `next_action_owner` is `human` and `status` is `blocked`.
- A scheduled cron trigger that summarises the day's task progress across many repos.
- A simple terminal dashboard that lists open tasks across many repos by globbing `~/code/*/.claude-task/*/STATUS.md`.
- A pre-`/clear` hook that refuses to clear if `ACTIVE` points at a task whose `STATUS` hasn't been updated in this session.

10. Anti-patterns to avoid

- **Writing to LOG.md without updating STATUS.md.** The orchestrator polls a stale snapshot. Every meaningful LOG entry should bump the STATUS `updated` timestamp at minimum.
- **Editing BRIEF.md after init.** It's write-once. If scope genuinely changes, add a DECISIONS entry ("scope expanded to include X — why: ...") and let BRIEF stand. This preserves the original intent for blame and audit.
- **Skipping checkpoints to save time.** That is exactly how context bloats and goal #2 fails.
- **Creating a task folder for trivial one-shot work.** Threshold: "expected to span more than one session, or produce non-trivial decisions worth preserving." A typo fix doesn't qualify.

- **Multiple agents editing the same task simultaneously.** No locking. Coordinate via `owner` and `session_id` in STATUS frontmatter.
- **Storing secrets in any of these files.** They're plaintext on disk; if you ever decide to commit, secrets leak.
- **Letting auto-memory carry per-task state.** Auto-memory is for slow-moving facts. Per-task working state belongs in `.claude-task/` .

11. Example: the live Foundation task

The first real use of the system was the in-flight UI redesign Foundation work in AdhocPrintStudio. The folder `.claude-task/ui-redesign-foundation/` was created with the init script, then populated by hand to capture the current state:

- `STATUS.md` : `status: awaiting_review` , owner flipped to human, frontmatter shows blockers_open: 0 and next_action_owner: human.
- `BRIEF.md` : goal, in/out-of-scope lists, success criteria, source-spec links to commits `f6b7642` (design) and `1f2ec8e` (plan).
- `NEXT.md` : literal command for plan Phase 0.1 — `npm install` the Radix and lucide packages — plus an "if blocked" fallback.
- `DECISIONS.md` : four entries explaining why this was decomposed as sub-project A, why Radix, why `apps/web/src/design/` , why the `--accent` rename.
- `ARTIFACTS.md` : design-spec commit, plan commit, v0.4.1 release URL, mockup HTML files.
- `LOG.md` : two entries, newest first.

A new agent opening AdhocPrintStudio in a fresh session, told nothing else, can read those four short files and know exactly where the work stands and what to do next.

12. Where things live (file map)

WHAT	WHERE
Templates and init script	`C:\Users\Eric\claude-task-templates\`
Public mirror	github.com/ericwang19832003/claude-task-templates
Design spec	`docs/specs/2026-04-20-design.md` in that repo
Implementation plan	`docs/specs/2026-04-20-plan.md` in that repo
Live example task folder	`C:\AdhocPrintStudio-main\AdhocPrintStudio-main\.claude-task\ui-redesign-foundation\` (local-only)
Per-repo committed README	`<any-repo>/.claude-task/README.md`
Per-task local working memory	`<any-repo>/.claude-task/<slug>/` (git-ignored)
Auto-memory (separate system)	`~/.claude/projects/ … /memory/`
Harness per-session tasks (separate system)	`~/.claude/tasks/<session>/<id>.json`

End of document. Generated 2026-04-20 from `docs/guide.html` in the claude-task-templates repo.